

UNIVERSIDAD NACIONAL AGRARIA LA MOLINA
FACULTAD DE ECONOMÍA Y PLANIFICACIÓN
DEPARTAMENTO ACADÉMICO DE ESTADÍSTICA E
INFORMÁTICA



Detección de estafas laborales en las páginas de Bumeran y
CompuTrabajo

Integrantes:

- Arroyo Arruz, Alejandra Margoth | 20211805
- Maguiña Tabory, Leonardo Fabio | 20241388
- Polo D'Arrigo, Salvatore Diego | 20241394
- Solis Leon, Nilo Andy | 20240734

Curso: Lenguaje de Programación II

Fecha de entrega: Domingo 28 de junio de 2026

Docente: Ana Cecilia Vargas Paredes

Lima – Perú

2026

Índice

1.	Introducción	1
2.	Planteamiento del problema	1
3.	Justificación.....	1
4.	Objetivos	2
	Objetivo general	2
	Objetivos específicos.....	2
5.	Descripción de la solución	2
	Scraping de la página Bumeran.....	2
	Scraping de la página TrabajosDiarios.....	3
6.	Diagrama de flujo del desarrollo	4
	Flujo del proceso	4
7.	Almacenamiento de los datos.....	5
8.	Resultados	5
	Gráfica 1: Frecuencia de ofertas según clasificación y página web.....	5
	Gráfica 2: Palabras más frecuentes en las páginas	6
9.	Conclusiones	7
10.	Reflexión sobre el uso de los LLMs.....	8

1. Introducción

Actualmente, la búsqueda de empleo por internet es una práctica común en el Perú. Plataformas como Bumeran y Trabajos Diarios ofrecen diversas oportunidades laborales; sin embargo, algunas publicaciones pueden presentar señales de alerta o información poco confiable.

Por ello, este proyecto utiliza Python y técnicas de WebScraping para recopilar, limpiar y analizar ofertas laborales. A partir del texto obtenido, se identifican palabras clave sospechosas y se asigna una clasificación según el puntaje de cada oferta.

De esta manera, el trabajo busca diferenciar entre ofertas no sospechosas, potencialmente sospechosas y sospechosas, mostrando cómo la programación puede apoyar el análisis de información laboral en internet.

2. Planteamiento del problema

Hoy en día, buscar trabajo en internet es algo completamente normal. Plataformas como **Trabajos Diarios** y **Bumeran** reciben miles de visitas al día de personas que están buscando empleo en el Perú. Sin embargo, no todas las ofertas que aparecen en estos portales son reales. Cada vez es más común encontrar publicaciones falsas que buscan engañar a quienes necesitan trabajo, aprovechándose de su situación.

Estas ofertas falsas suelen tener características en común: prometen sueldos muy altos, indican que no se necesita experiencia, afirman que el trabajo es sencillo, no mencionan el nombre de la empresa o solicitan que el postulante realice algún pago antes de ser contratado. Caer en este tipo de engaños puede generar pérdidas de dinero y tiempo para quienes buscan empleo.

Por ello, se realiza la siguiente pregunta: **¿De qué manera un programa de Python que realice un recuento de palabras en una oferta laboral puede detectar de manera efectiva si tal oferta es una estafa?**

3. Justificación

La búsqueda de empleo mediante plataformas digitales ha aumentado notablemente en los últimos años, facilitando el acceso a miles de ofertas laborales. Sin embargo, este crecimiento también ha favorecido la aparición de ofertas falsas o engañosas que buscan aprovecharse de las personas que buscan trabajo.

Detectar este tipo de publicaciones de manera manual resulta difícil debido a la gran cantidad de ofertas que se publican cada día. Por ello, es necesario desarrollar herramientas que permitan identificar automáticamente aquellas ofertas que presentan características sospechosas, contribuyendo a que los usuarios puedan tomar decisiones más informadas antes de postular.

En este contexto, el uso de Python y técnicas de WebScraping permite recopilar y analizar información de portales de empleo como Bumeran y Trabajos Diarios. Mediante el análisis de palabras clave y criterios previamente definidos es posible clasificar las ofertas según su nivel de sospecha, demostrando cómo la programación puede contribuir a resolver un problema real mediante el procesamiento automatizado de información disponible en internet.

4. Objetivos

Objetivo general

Desarrollar una aplicación en Python que utilice técnicas de WebScraping para recopilar y analizar ofertas laborales publicadas en Bumeran y Trabajos Diarios, con el fin de identificar posibles señales de fraude y clasificarlas según su nivel de sospecha.

Objetivos específicos

- Recopilar al menos 20 ofertas laborales de cada portal utilizando un programa desarrollado en Python.
- Identificar señales de alerta en cada oferta, como sueldos excesivamente altos, ausencia del nombre de la empresa, falta de experiencia requerida o solicitudes de pagos previos.
- Clasificar automáticamente cada oferta en una de las categorías "Sospechosa", "Potencialmente sospechosa" o "No sospechosa", de acuerdo con las características detectadas.
- Almacenar los resultados obtenidos en un archivo CSV para facilitar su revisión, análisis y comparación.

5. Descripción de la solución

Inicialmente se crea una lista extensa llamada “listapalabras” que contiene las palabras sospechosas o keywords a contabilizar en cada oferta laboral de ambas páginas.

Scraping de la página Bumeran

Se importan los paquetes: json, bs4, pandas y requests.

Se realizará un scrapeo de diversos archivos .json extraídos de la API de la misma página. Los archivos pertenecen a diversas páginas de ofertas laborales tomadas aleatoriamente como muestra. El código siguiente lee los archivos:

```
for i in range(0,len(url)):
    with open("empleo" + str(i) + ".json", encoding="utf-8") as f:
        data = json.load(f)
```

```
descripcion_html = data["aviso"]["descripcion"]
```

```
textoagregar = BeautifulSoup(
    descripcion_html,
    "html.parser"
).get_text(" ", strip=True)
texto.append(textoagregar)
```

Por consiguiente, se ejecuta el siguiente código para crear el DataFrame final:

```
datospd = {"Fuentes/Palabras": ["Página"] + listapalabras +
["Puntaje","Clasificación", "URL"]}
```

Se manipula el diccionario datospd tal que se pueda ejecutar lo siguiente:

```
DataF = pd.DataFrame(datospd)
DataFinal = DataF.transpose()
DataFinal
```

El resultado final debe ser el siguiente:

		0	1	2	3	4	5	6	7	8	9	...	160	161	162	163	164	165
Fuentes/Palabras	Página	sin experiencia	con o sin experiencia	pago	pagado	pago diario	pago semanal	ingresos	ingresos extras	ingresos adicionales	—	potencial ilimitado	oportunidad de crecimiento	rápida incorporación	inmediato	postulación inmediata	vacante urgente	demanda
Fuente1	BUMERAN	0	0	0	0	0	0	0	0	0	0	—	0	0	0	0	0	0
Fuente2	BUMERAN	0	0	1	0	0	0	3	0	0	0	—	0	0	0	0	0	0
Fuente3	BUMERAN	0	0	0	0	0	0	0	0	0	0	—	0	0	0	0	0	0
Fuente4	BUMERAN	1	0	0	0	0	0	0	0	0	0	—	0	0	0	0	0	0
Fuente5	BUMERAN	0	0	0	0	0	0	0	0	0	0	—	0	0	0	0	0	0
Fuente6	BUMERAN	0	0	0	0	0	0	0	0	0	0	—	0	0	0	0	0	0
Fuente7	BUMERAN	0	0	0	0	0	0	0	0	0	0	—	0	0	0	0	0	0
Fuente8	BUMERAN	0	0	0	0	0	0	0	0	0	0	—	0	0	0	0	0	0
Fuente9	BUMERAN	0	0	1	0	0	0	0	0	0	0	—	0	0	0	0	0	0
Fuente10	BUMERAN	0	0	0	0	0	0	0	0	0	0	—	0	0	0	0	0	0

Scraping de la página TrabajosDiarios

Se importan los paquetes: re y time.

Se crea una lista llamada "url" que contendrá la muestra elegida de ofertas de la página. Se crea el diccionario respectivo:

```
datospd = {"Fuentes/Palabras": ["Página"] + listapalabras +
["Puntaje", "Clasificación", "URL"]}
```

Se ejecuta el siguiente código:

```
for i in range(len(urls)):
    numero = i + 1
    try:
        headers = {
            "User-Agent": "Proyecto de Web Scraping - Python Requests"
        }

        resp = requests.get(urls[i], headers=headers, timeout=25)

        with open(f"oferta{numero}.json", "w", encoding="utf-8") as f:
            json.dump({"aviso": {"descripcion": resp.text}}, f,
ensure_ascii=False)

        with open(f"oferta{numero}.json", encoding="utf-8") as f:
            data = json.load(f)

        sopa = BeautifulSoup(resp.text, "html.parser")

...
...
...

        time.sleep(4.5)

except Exception as e:
    print(f"se encontró un error en la URL {numero}: {e}")
```


7. Almacenamiento de los datos

Luego de realizar la extracción y limpieza de la información, los datos obtenidos fueron organizados en estructuras tipo dataframe. Cada página web trabajada generó su propio conjunto de datos, donde se almacenó información como la página de procedencia, las palabras clave encontradas, el puntaje total, la clasificación asignada y la URL de la oferta laboral.

En primer lugar, los datos extraídos de cada oferta fueron convertidos en dataframes individuales. Esto permitió ordenar la información en filas y columnas, facilitando su análisis posterior. Posteriormente, los dataframes obtenidos de Bumeran y Trabajos Diarios fueron combinados en un único dataframe general denominado `DataFrame_Final`.

Para integrar la información se utilizó la función `pd.concat()`, la cual permite unir varios dataframes en una sola estructura de datos. Después de la unión, se reinició el índice para mantener el orden de los registros y se eliminaron las filas innecesarias, en caso de existir.

Finalmente, los resultados fueron exportados en formato CSV para conservar la información procesada y facilitar su uso en herramientas como Excel, Google Sheets o Python. El archivo principal generado fue `DatosFinales.csv`, el cual reúne toda la información consolidada obtenida durante el desarrollo del proyecto.

A continuación, se muestra el código respectivo:

```
DataFrame_Final      =      pd.concat([DataFinal,      DataFinal_b],
ignore_index=True)
DataFrame_Final      =
DataFrame_Final.drop(index=21).reset_index(drop=True)
DataFinal.to_csv("DatosBumeran.csv", header=False)
DataFinal_b.to_csv("DatosTrabajosDiarios.csv", header=False)
DataFrame_Final.to_csv("DatosFinales.csv",      header=False,
index=False)
DataFrame_Final
```

8. Resultados

Para los resultados, se procedió a realizar unas gráficas para facilitar con la visualización de datos. Los resultados fueron los siguientes:

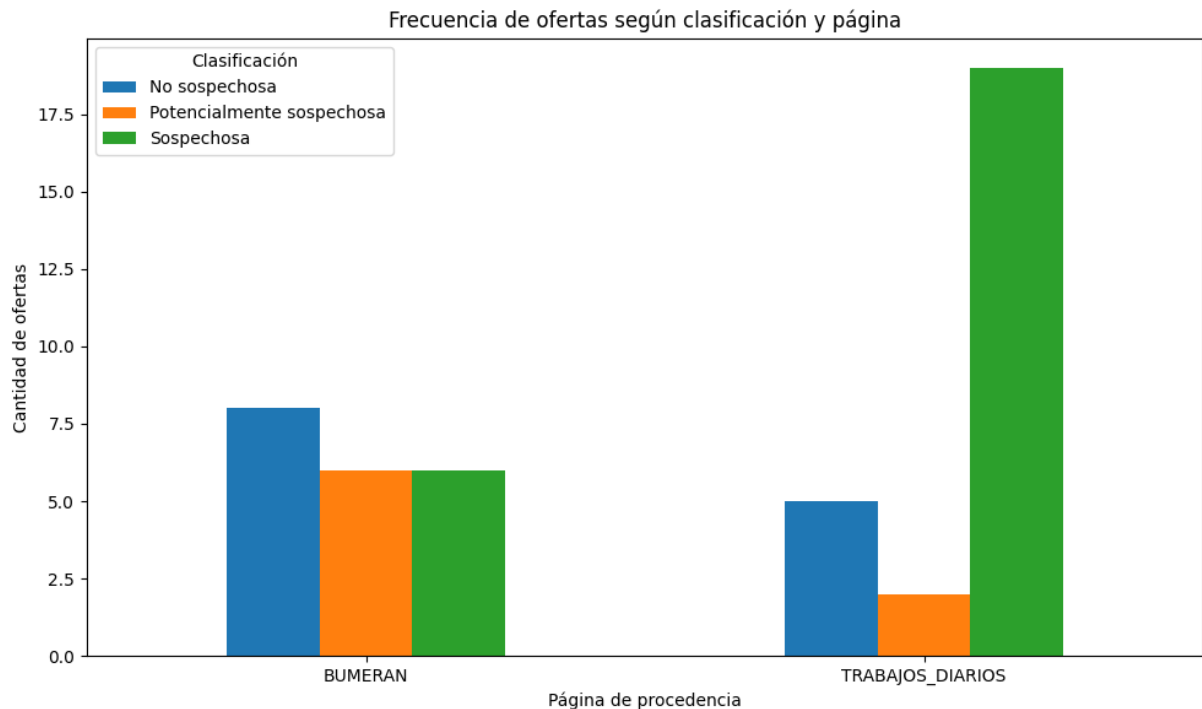
Gráfica 1: Frecuencia de ofertas según clasificación y página web

El código utilizado fue el siguiente:

```
tabla_clasificacion = pd.crosstab(
    df_graf["Página"],
    df_graf["Clasificación"]
)
tabla_clasificacion.plot(kind="bar", figsize=(10, 6))
plt.title("Frecuencia de ofertas según clasificación y página")
plt.xlabel("Página de procedencia")
plt.ylabel("Cantidad de ofertas")
plt.xticks(rotation=0)
plt.legend(title="Clasificación")
plt.tight_layout()
```

```
plt.show()
```

La gráfica respectiva es:



Interpretación: Se logra apreciar que la página Bumeran mantiene de forma equilibrada las noticias por su clasificación, mientras que TrabajosDiarios no mantiene este equilibrio, teniendo una cantidad considerablemente mayoritaria de ofertas laborales sospechosas.

Gráfica 2: Palabras más frecuentes en las páginas

Se hará recuento de las 10 palabras sospechosas más frecuentes en las muestras elegidas de cada página web.

Se hizo uso del siguiente código:

```
columnas_no_palabras = ["Página", "Puntaje", "Clasificación", "URL"]
```

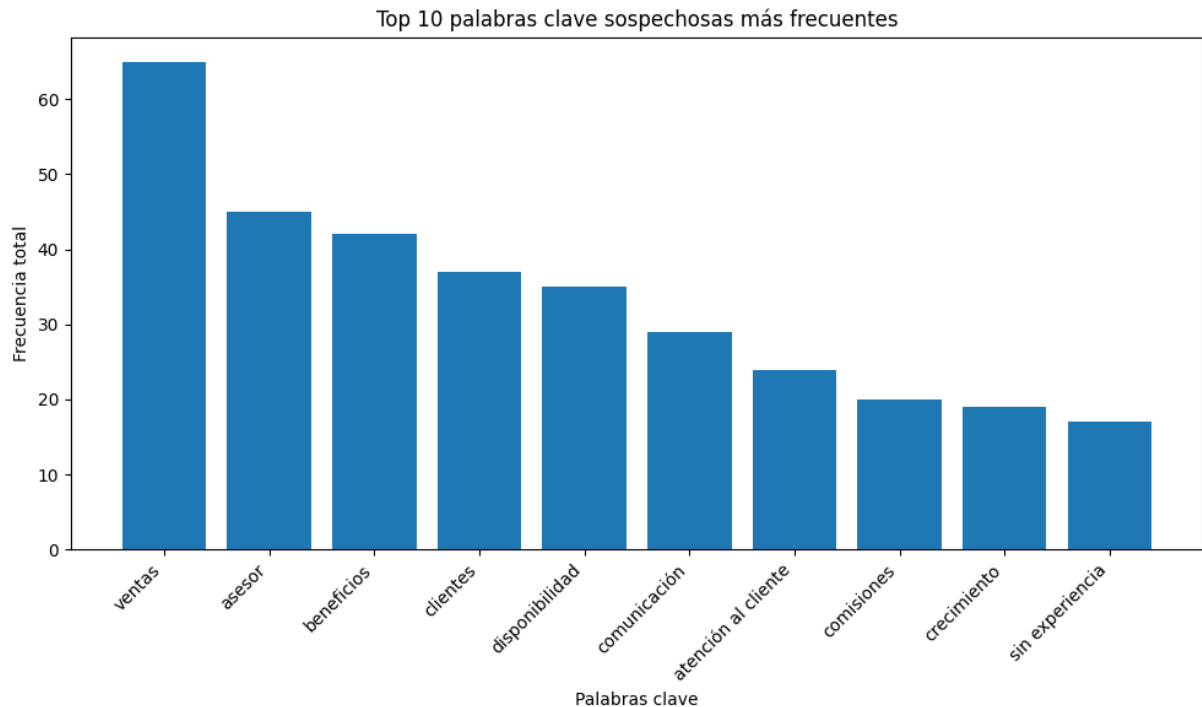
```
columnas_palabras = [  
    columna for columna in df_graf.columns  
    if columna not in columnas_no_palabras  
]  
df_palabras = df_graf[columnas_palabras].apply(pd.to_numeric,  
errors="coerce").fillna(0)  
top_10_palabras = df_palabras.sum().sort_values(ascending=False).head(10)  
plt.figure(figsize=(10, 6))  
plt.bar(top_10_palabras.index.astype(str), top_10_palabras.values)  
plt.title("Top 10 palabras clave sospechosas más frecuentes")  
plt.xlabel("Palabras clave")  
plt.ylabel("Frecuencia total")  
plt.xticks(rotation=45, ha="right")
```



```
plt.tight_layout()
```

```
plt.show()
```

La gráfica respectiva sale:



Interpretación: Se logra apreciar que en las muestras de ofertas de Bumeran y Computrabajos, las palabras sospechosas más frecuentes tienen que ver con “ventas”, “asesor” y “beneficios”, indicando que por lo general, en las estafas laborales frecuentan a usar palabras que contengan un índole llamativo con respecto a las utilidades.

9. Conclusiones

- El proyecto permitió aplicar técnicas de WebScraping para recopilar información de ofertas laborales publicadas en Bumeran y Trabajos Diarios. A partir de esta información, se desarrolló una metodología simple para identificar posibles señales de alerta en publicaciones de empleo.
- Los datos obtenidos fueron limpiados, organizados y convertidos en dataframes. Posteriormente, los resultados de ambas páginas fueron combinados en un solo dataframe llamado `DataFrame_Final`, lo que permitió analizar toda la información de manera conjunta.
- Las gráficas elaboradas permitieron observar la frecuencia de ofertas según su clasificación y página de procedencia. Además, facilitaron la identificación de las palabras clave sospechosas que aparecieron con mayor frecuencia en las publicaciones laborales analizadas.
- El uso de palabras clave y puntajes permitió clasificar las ofertas en tres categorías: No sospechosa, Potencialmente sospechosa y Sospechosa. Esta clasificación facilita una primera revisión automática de publicaciones que podrían presentar señales de riesgo.
- Finalmente, el uso de inteligencia artificial fue de apoyo durante el desarrollo del proyecto, especialmente en la programación, limpieza de texto, solución de errores y

elaboración de gráficos. Sin embargo, fue necesario revisar y validar los resultados obtenidos para garantizar que el análisis fuera correcto y consistente.

10. Reflexión sobre el uso de los LLMs

El uso de modelos de lenguaje basados en inteligencia artificial fue una herramienta de apoyo importante durante el desarrollo del proyecto. La IA contribuyó principalmente en la elaboración del código, la corrección de errores, la limpieza de texto y la organización del flujo de trabajo.

Durante el proceso de WebScraping, la IA brindó apoyo en el uso de herramientas como requests, BeautifulSoup y expresiones regulares. Asimismo, facilitó la comprensión de cómo extraer información desde archivos JSON y eliminar contenido innecesario de las páginas analizadas.

Además, la IA fue de utilidad para interpretar el archivo robots.txt, permitiendo comprender las restricciones de acceso establecidas por cada sitio web e identificar las rutas que debían evitarse durante la extracción de datos.

También se empleó como apoyo en la elaboración de las gráficas finales, colaborando en la preparación del dataframe, la construcción de tablas de frecuencia y la visualización de las palabras clave sospechosas identificadas en las ofertas laborales.

Sin embargo, la IA no reemplazó el trabajo realizado por el grupo. Fue necesario revisar el código, ejecutar las pruebas correspondientes, corregir errores, adaptar las soluciones a las necesidades del proyecto y verificar que los resultados obtenidos fueran coherentes. En consecuencia, la inteligencia artificial funcionó como una herramienta de apoyo, mientras que la validación y las decisiones finales estuvieron a cargo de los integrantes del equipo.